

IncidentLab: Multimodal Incident Detection and Evidence-Supported Investigation with CPU Serving and Kubernetes Validation

Amikula Pavan Kumar Goud
MSc Computer Science Student
Blekinge Institute of Technology
Karlskrona, Sweden

Abstract— IncidentLab connects telemetry processing, machine learning, chronological inference, and evidence-supported investigation in a reproducible cloud incident project. The research pipeline processed approximately 49.7 million log records and 110.7 million spans from 733 eligible RCAEval cases, partitioned into 439 training, 147 validation, and 147 test cases. A temporal gated recurrent unit achieved window F1 of 0.972 on the development-test partition, compared with 0.842 for Isolation Forest. External transfer was weak and is retained as a negative finding. A separate CPU deployment experiment collected real requests from frontend, checkout, and inventory services running in a local kind Kubernetes cluster. Operational warnings flagged all nine held-out controlled fault trials; the deployment-specific Isolation Forest flagged four. Trace review confirmed trial impact, while capture gaps and proxy timing labels limit window-level interpretation. The project demonstrates an executable research-to-serving workflow rather than production reliability or automated causal diagnosis.

Keywords—incident detection; multimodal telemetry; GRU; Isolation Forest; distributed tracing; CPU inference; Kubernetes.

1. INTRODUCTION

Cloud service failures often affect several applications because requests traverse dependencies. A slow inventory operation can also increase checkout and frontend latency. Conversely, a fast failed response can reduce latency while increasing errors. Detection therefore requires measurements of both timing and outcomes, and investigation requires request relationships rather than an unsupported service ranking.

IncidentLab combines a research evaluation with an executable application testbed. Its contributions are (1) processing and case-separated evaluation of multimodal telemetry; (2) preserved conventional and temporal model experiments, including external negative results; (3) CPU serving and durable chronological observation; and (4) measured fault trials on real HTTP services and an actual

local Kubernetes deployment. Operational warnings and ML decisions are reported separately throughout. This is a student project report in IEEE conference style, not a claim of publication or industrial deployment.

2. DATA AND RESEARCH PROTOCOL

RCAEval processing

RCAEval supplies microservice root-cause evaluation cases and telemetry [1]. The acquired collection contained 735 cases; 733 were eligible for the existing processing workflow. Approximately 49.7 million log records and 110.7 million spans were processed into portable inputs. The supplied project preserves acquisition scripts, preprocessing code, source provenance, trained checkpoints, predictions, and reports. Large raw collections remain on the original college computer.

Training, validation, and test partitions contained 439, 147, and 147 cases respectively. Case separation avoids treating windows from the same incident as independent partition examples. Validation selected model settings and thresholds; saved test predictions were used for reporting. The evaluation uses the project's existing labels and processing definitions. Correlated windows, benchmark-specific telemetry, and a development-test partition constrain generalization claims.

Models and negative findings

Conventional experiments include Isolation Forest and a supervised boosting reference. Isolation Forest isolates observations through randomized trees [2]. The PyTorch temporal model uses a gated recurrent unit over multimodal telemetry features, enabling sequential information to influence anomaly scoring. Its saved checkpoint has 15,845 parameters and was selected at epoch 9 from 12 recorded training epochs. The original GPU training was not repeated on the laptop.

External AnoMod evaluation exposed weak transfer: the frozen GRU alerted on 3 of 24 fault runs and the boosting reference on 2 of 24. Independent healthy examples and exact onset labels were unavailable for that evaluation, so

these run flags cannot establish precision, F1, or production recall. OpenStack parameter-based models also produced false alarms. Optional local LLM explanation experiments contained unsupported statements even among accepted outputs. Saved outputs remain available, but new LLM generation is disabled and automatic remediation is absent.

3. ARCHITECTURE AND SERVING

Application and telemetry path

Three real HTTP services form the chain frontend → checkout → inventory. Request events record timestamps, durations, HTTP outcomes, service names, trace identifiers, span identifiers, and parent relationships. Three-second windows summarize captured requests. p95 latency describes the duration below which 95% of captured requests finished; it is not a fault probability. An upstream duration includes time waiting for downstream responses.

The FastAPI dashboard exposes saved research replay, measured experiment evidence, and a bounded live localhost demo. The demo launches owned services, sends requests, and performs frozen model inference. It does not launch research training. Saved results remain viewable after a collection ends. Current Kubernetes pod readiness is displayed separately from archived trial charts so a running pod is not mistaken for an active telemetry collection.

Durable observation and deployment

SQLite observation state persists request evidence and collector progress, rejects duplicate span events, and retains arrival cutoffs for chronological replay. Capture snapshots record pod identity and lifecycle. Overlapping log polls deduplicate request events, but deletion or log rotation can remove records before collection. Missing telemetry therefore requires capture review before being described as an application outage. Kubernetes logging behavior is relevant to this limitation [4].

The laptop deployment uses kind, which runs local Kubernetes nodes in Docker containers [3]. The dedicated context is kind-incidentlab; the owned namespace is incidentlab-testbed. The node container is incidentlab-control-plane. A single node was configured with a two-CPU, 4 GiB limit. In this configuration the collector, inference process, and dashboard run on Windows; the three HTTP applications run in Kubernetes pods. The research GRU is not deployed as a pod or retrained for the cluster experiment.

Detectors and investigation

A small native Isolation Forest is fitted to fresh healthy deployment windows. Separate calibration windows set its threshold, after which it remains frozen for held-out controls and faults. Operational warnings additionally use latency and error envelopes and coverage checks. These mechanisms can flag behavior missed by ML. Evidence grouping and investigation inspect dependent service measurements and request traces; an alert or ranking alone is not proof of a root cause.

For binary window decisions, precision is $TP/(TP+FP)$, recall is $TP/(TP+FN)$, and $F_1=2TP/(2TP+FP+FN)$. Trial detection instead asks whether a controlled trial was flagged. A trial flag and a grouped incident are different units. Continuous-score ROC and precision–recall curves are generated separately for each research system to avoid pooling incompatible score scales.

4. RESEARCH RESULTS

Table I summarizes preserved RCAEval results. The GRU confusion counts in Fig. 1 cover 2,535 test windows: 881 true negatives, 19 false positives, 72 false negatives, and 1,563 true positives. Its high development-test performance does not override the weak external transfer. Figure 2 reproduces recorded training loss and validation F1, with no new fitting.

Preserved RCAEval development-test results

Model	Precision	Recall	F1
Isolation Forest	0.972	0.743	0.842
Temporal GRU	0.988	0.956	0.972

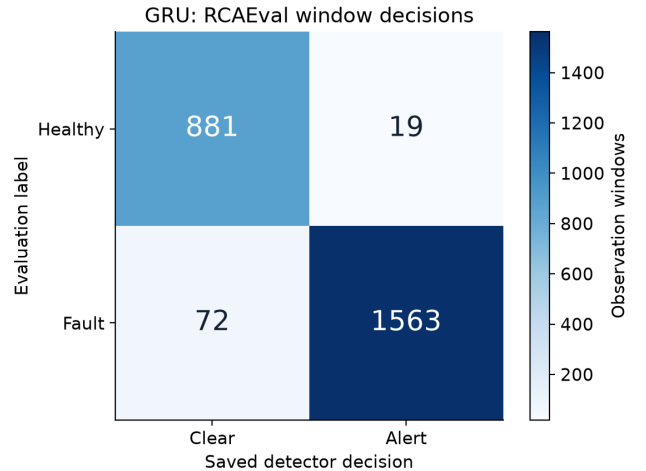


Fig. 1. GRU development-test window confusion matrix. Rows are saved evaluation labels; columns are thresholded decisions. Windows within a case are correlated.

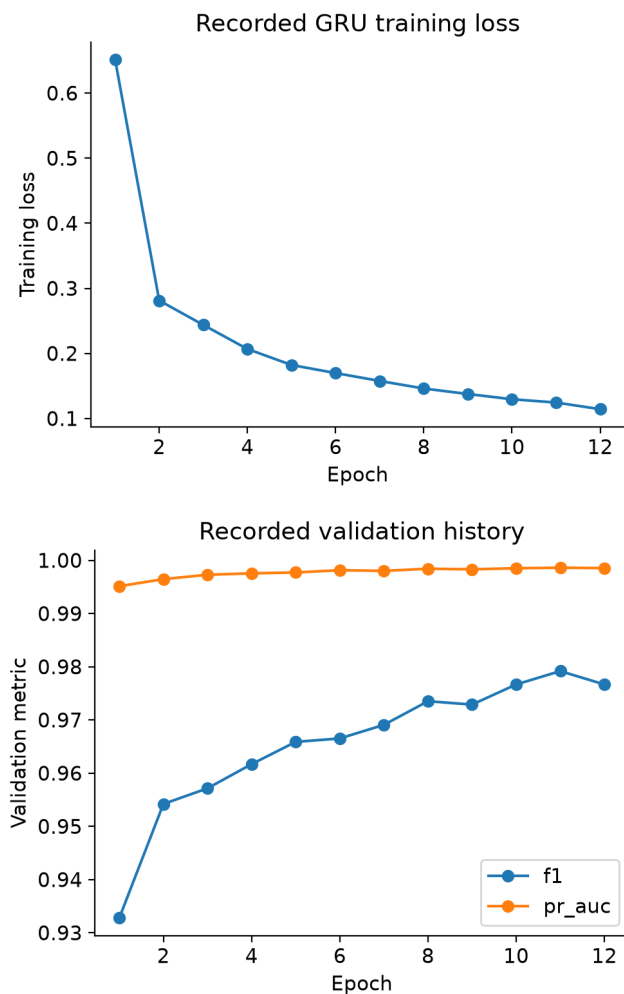


Fig. 2. Original 12-epoch GRU history. The selected checkpoint was epoch 9; laptop execution reuses it. Loss and F1 have different units.

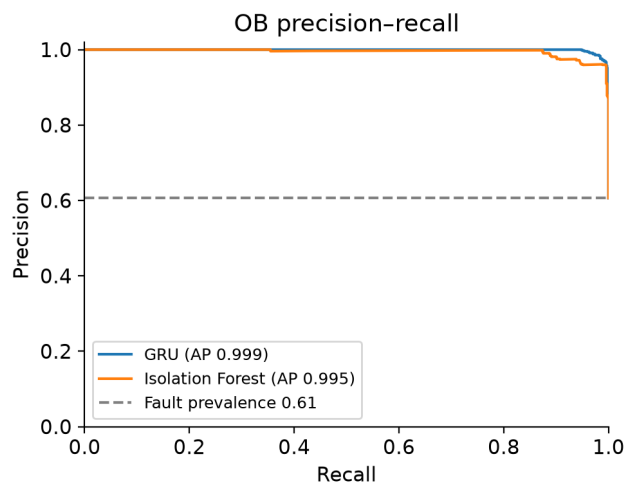
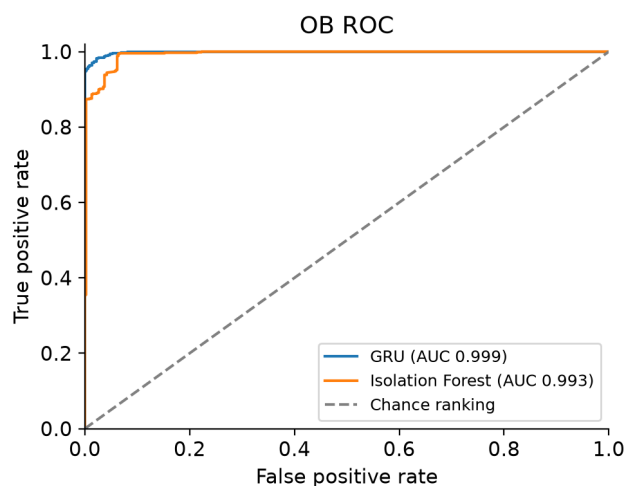


Fig. 3. OB continuous-score ROC and precision-recall curves from saved test predictions. Scores are compared within this system; no test threshold tuning.

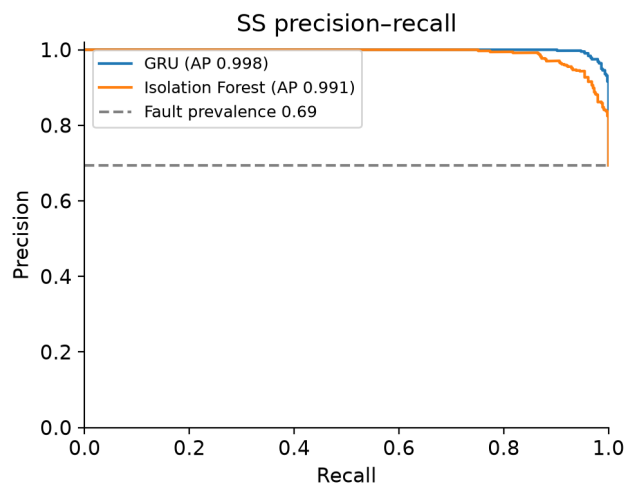
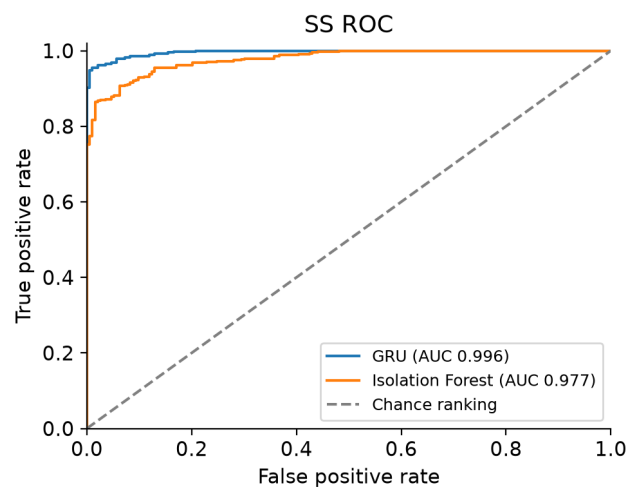


Fig. 4. SS continuous-score ROC and precision-recall curves from saved test predictions. Scores are compared within this system; no test threshold tuning.

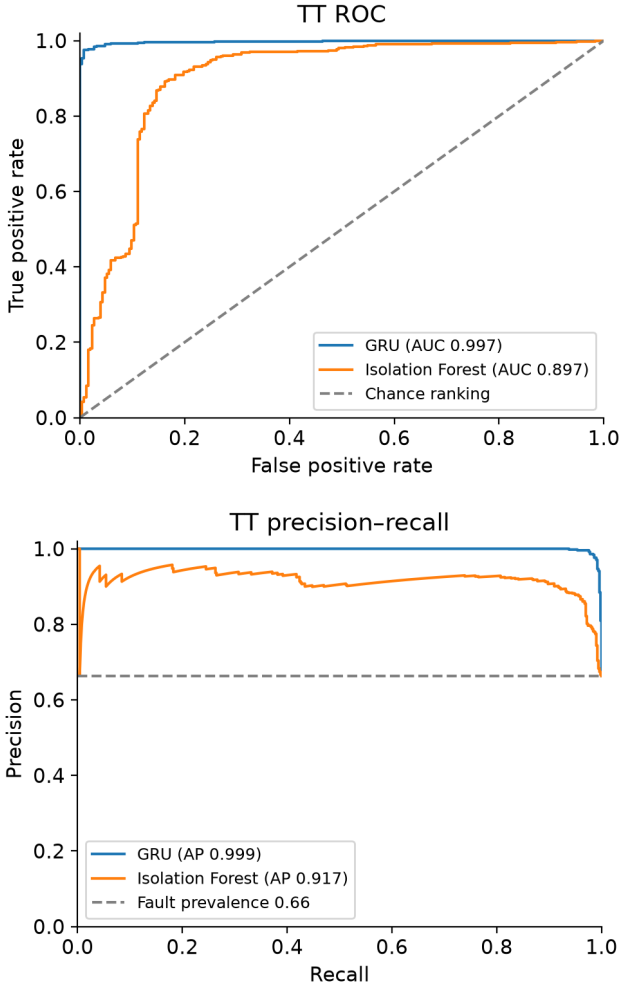


Fig. 5. TT continuous-score ROC and precision-recall curves from saved test predictions. Scores are compared within this system; no test threshold tuning.

5. MEASURED APPLICATION EXPERIMENTS

Native HTTP experiment

The original native experiment comprised 22 phases and 1,140 observation windows, including 18 controlled trials across three services. Its operational detection and grouping approach detected all 18 trials; Isolation Forest alone detected 10. Healthy training and calibration were separate and the dedicated checkpoint was frozen before evaluation. These are measured local HTTP experiments, not company-production results. Their deployment and calibration differ from the cluster experiment, so their counts are not a paired improvement comparison.

Held-out Kubernetes validation

The completed laptop validation contains 13 phases and 420 windows: 60 training, 60 calibration, and 300 scored held-out windows. It recorded 3,413 client requests and 9,680 captured spans, with zero dropped workload requests. Controls include three-minute healthy and benign surge phases. Delay, HTTP error, and service unavailability were injected for each of the three services, producing nine controlled fault trials.

Operational trial flags occurred in 9 of 9 trials and ML-only flags in 4 of 9 (Table II). Trace review confirmed impact in all nine trials. Healthy control requests all returned HTTP 200 with zero operational or ML warnings. Surge requests also all returned HTTP 200; operational warnings remained zero, but three ML windows warned. A separate ML warning after delay recovery is retained as a false alarm rather than credited as fault detection.

Trial flags: retain detector and deployment boundaries

Experiment	Operational	ML only
Native HTTP (college PC)	18/18	10/18
Kubernetes (laptop)	9/9	4/9

Figure 6 shows inventory delay propagating upstream. All delay trials were missed by the cluster ML detector, while operational latency checks flagged them. During checkout errors, both checkout and frontend failed, and inventory received no requests because the chain stopped early. During frontend errors, downstream services received no requests. Low latency during these errors means fast failure, not improved service quality. Recovery windows returned to healthy measurements after the injected fault was cleared.

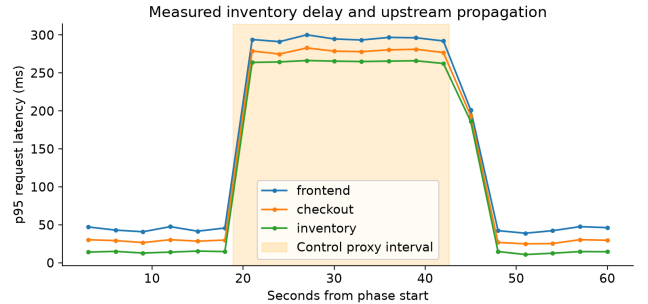


Fig. 6. Captured request latency during the held-out inventory delay phase. Lines show measured windows, not a predicted causal explanation.

Evidence and resource checks

All 300 scored windows were replayed from arrival-bounded captured evidence and the frozen deployment checkpoint. Feature, score, and decision mismatches were zero. Control-command completion timestamps remain onset and recovery proxies; boundary-crossing windows are excluded from proxy-window matrices. Trace review validates trial impact, not independently adjudicated labels for every window.

Twenty-one successful request chains lacked spans near pod deletion, with 14 missing parent links. Stored events were deduplicated, but deduplication cannot recover events never captured. The results preserve these limitations and earlier failed bootstrap, HTTP framing, and termination-timing attempts. A separate infrastructure check verified one same-pod Inventory container restart: all eight unpolled Inventory trace tails were recovered from previous-container logs, repeated polling inserted zero duplicates, and all 32 check requests succeeded. Two unsuccessful attempts to trigger a restart are preserved. This bounded check does not establish log-rotation or repeated-restart completeness and does not change the nine-trial metrics.

Measured CPU inference p95 was 52.52 ms; window-close-to-decision p95 was 6.28 s, including the collection and scheduling path. Fifty resource samples observed a node memory maximum of 928 MiB, Windows available memory minimum of 2.08 GiB, and disk free minimum of 19.99 GiB. These are sampled observations, not true peak requirements or guarantees for other laptops.

6. REPRODUCIBILITY AND REMAINING VALIDATION

The transfer originally verified 2,502 file hashes. A clean CPU configuration loads saved checkpoints using PyTorch 2.11.0+cpu. CPU/GPU prediction differences were below 2×10^{-6} in portability checks. Source amendments are documented separately from original provenance. The lean CPU test suite passes 75 tests; the optional transformer diagnosis module is excluded. The user interface and figure builder read saved predictions and include source hashes; exported PNG, SVG, PDF, and JSON artifacts support report reuse.

Inspect TRANSFER_README.md, laptop handoff notes, and deployment documentation before execution. Laptop setup and startup use Setup-Laptop.ps1 and Start-Laptop.ps1; the amended workspace wrapper records source changes. Cluster modes are Check, Deploy, Quick, and Validate in Run-Cluster.ps1. Results use separate run directories under artifacts/kubernetes. The reproducible execution report identifies the validated run as 20261006T141939-0fc666. Docker must remain running for actual kind workloads; saved dashboard replay and the native localhost demo do not require the cluster engine.

Remaining industrial validation includes independent external holdouts with defensible healthy/onset labels, long-duration operation, log rotation and repeated restart capture, authenticated multi-user serving, and larger deployment evaluation. The local cgroup compatibility configuration and one-second termination grace are explicit testbed choices. Their suitability for production has not been established. Unsupported LLM explanations, deployment false alarms, and collection failures must remain visible rather than removed to improve reported performance.

7. CONCLUSION

IncidentLab demonstrates real telemetry processing, temporal and conventional ML evaluation, CPU checkpoint serving, durable observation, Docker/kind deployment, controlled fault detection, and trace-supported investigation. Strong RCAEval development-test metrics coexist with weak external transfer. Operational checks detect controlled failures missed by ML; neither result establishes universal reliability. The completed laptop demonstration is reproducible and useful for explaining engineering decisions, while industrial and causal validation remain explicit future work.

REFERENCES

- [1] L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," arXiv:2412.17015, 2024, revised 2025. doi: 10.48550/arXiv.2412.17015.
- [2] Scikit-learn developers, "IsolationForest," official API documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>. Accessed: Oct. 6, 2026.
- [3] Kubernetes SIGs, "kind: Kubernetes in Docker," official documentation. [Online]. Available: <https://kind.sigs.k8s.io/>. Accessed: Oct. 6, 2026.
- [4] Kubernetes authors, "Logging architecture," official documentation. [Online]. Available: <https://kubernetes.io/docs/concepts/cluster-administration/logging/>. Accessed: Oct. 6, 2026.

PDF rendering note: this readable two-column version was generated with ReportLab because the built-in LaTeX compiler returned a platform-directory error. The companion IEEEtran source is the editable submission-format source. No venue compliance or publication is claimed.